

TODITO PAGOS, S.A. DE C.V.
INSTITUCIÓN DE FONDOS DE PAGO
ELECTRÓNICO

Documento de Especificación para
Prueba Técnica: Integración API
Marvel

1. Objetivo

Desarrollar una aplicación web que consuma la API de Marvel para mostrar información de cómics, películas y personajes. La aplicación debe:

- Usar Node.js con Fastify en backend, con arquitectura en capas.
- Usar Vue.js con Vuex en frontend, siguiendo Domain-Driven Design (DDD).
- Implementar autenticación con login y manejo de múltiples cuentas.
- Guardar credenciales en un archivo .txt de forma segura.
- Permitir filtrado múltiple en tres pantallas: cómics, películas y personajes.
- Mostrar información general y la imagen de referencia en cada pantalla.
- Resolver al menos 3 de los 10 principales riesgos de TOP 10 OWASP.
- Seguir el modelo Cliente-Servidor.

Puntos extras:

- Realizar un doble factor de autenticación en el login, y al crear una nueva cuenta.

2. Arquitectura

a. Backend: Arquitectura en Capas

- **Capa de presentación (API REST con Fastify):** Controladores que reciben las peticiones HTTP.
- **Capa de aplicación:** Lógica de negocio, casos de uso.
- **Capa de dominio:** Entidades y reglas de negocio.
- **Capa de infraestructura:** Acceso a la API de Marvel, manejo de archivos (credenciales).

Ejemplo estructura de carpetas backend:

```
/backend  
/src  
  /controllers  
    authController.js  
    marvelController.js  
  /application  
    authService.js  
    marvelService.js  
  /domain  
    user.js  
    marvelEntity.js  
  /infrastructure  
    marvelApiClient.js  
    credentialStore.js  
  /routes  
    authRoutes.js  
    marvelRoutes.js  
server.js
```

b. Frontend: Arquitectura Domain-Driven Design (DDD)

- **Dominios:** Comics, Movies, Characters.
- **Componentes:** Cada dominio tiene sus componentes, store (Vuex), servicios.
- **Vista:** Pantallas para cada dominio con filtros y selección múltiple.

Ejemplo estructura de carpetas frontend:

```
/frontend
/src
  /domains
    /comics
      ComicsList.vue
      comicsStore.js
      comicsService.js
    /movies
      MoviesList.vue
      moviesStore.js
      moviesService.js
    /characters
      CharactersList.vue
      charactersStore.js
      charactersService.js
  /components
    FilterMultiSelect.vue
    Login.vue
  /store
    index.js
  /router
    index.js
  App.vue
  main.js
```

3. Ejemplos de Código

a. Backend

Server.js

```
const Fastify = require('fastify');
const authRoutes = require('./routes/authRoutes');
const marvelRoutes = require('./routes/marvelRoutes');

const fastify = Fastify({ logger: true });

fastify.register(authRoutes, { prefix: '/auth' });
fastify.register(marvelRoutes, { prefix: '/marvel' });

const start = async () => {
  try {
    await fastify.listen(3000);
    console.log('Server running on http://localhost:3000');
  } catch (err) {
    fastify.log.error(err);
    process.exit(1);
  }
};

start();
```

infrastructure/marvelApiClient.js

```
const axios = require('axios');

class MarvelApiClient {
  constructor(publicKey, privateKey) {
    this.publicKey = publicKey;
    this.privateKey = privateKey;
    this.baseUrl = 'https://gateway.marvel.com/v1/public';
  }

  _getAuthParams() {
    const ts = new Date().getTime();
    const hash = require('crypto')
      .createHash('md5')
      .update(ts + this.privateKey + this.publicKey)
      .digest('hex');
    return { ts, apikey: this.publicKey, hash };
  }

  async getComics(filters = {}) {
    const params = { ...this._getAuthParams(), ...filters };
  }
}
```

```
const response = await axios.get(`${this.baseUrl}/comics`, { params });
return response.data.data.results;
}

// Métodos similares para personajes y películas (series)
}

module.exports = MarvelApiClient;
```

infrastructure/credentialStore.js

b. Frontend

domains/comics/comicsStore.js
domains/comics/ComicsList.vue

4. Seguridad y OWASP Se espera que la solución aborde al menos 3 de los siguientes riesgos OWASP Top 10:

- A1: Broken Access Control: Implementar roles y permisos para que cada usuario solo acceda a sus datos.
- A2: Cryptographic Failures: Guardar credenciales de forma segura, no en texto plano (idealmente usar cifrado).
- A5: Security Misconfiguration: Configurar correctamente Fastify y CORS para evitar exposición innecesaria.
- A7: Identification and Authentication Failures: Implementar login seguro con manejo de sesiones o JWT.
- A10: Server-Side Request Forgery (SSRF): Validar y sanitizar las peticiones a la API externa.

5. Modelo Cliente-Servidor

- El frontend (cliente) consume la API REST del backend.
- Backend se encarga de la lógica, autenticación y comunicación con la API Marvel.
- Comunicación mediante JSON sobre HTTP.

6. Consideraciones Finales

- La lógica de negocio debe estar separada de la infraestructura.
- El frontend debe ser modular y escalable.
- Se debe documentar la API y los componentes.
- Se recomienda usar variables de entorno para las claves Marvel.

Documentacion de marvel:

<https://developer.marvel.com/>

Instrucción de entrega de texto libre:

Además de desarrollar el código solicitado en el ejercicio, deberán redactar en **media cuartilla** un documento en el que expliquen lo siguiente:

1. Explicación del código

- a. Describir brevemente qué hace el programa.
- b. Explicar de manera general la lógica que siguieron para resolver el problema.

2. Justificación de decisiones técnicas

- a. Indicar por qué eligieron ciertos métodos, algoritmos, técnicas criptográficas u otras herramientas para lograr la solución.
- b. Señalar qué ventajas o beneficios vieron en la implementación elegida.

3. Mejoras de seguridad (OWASP Top 10)

- a. Explicar cómo abordaron al menos **3 vulnerabilidades** de las 10 propuestas en el **OWASP Top 10** (por ejemplo: inyección, manejo de sesiones, validación de entradas, etc.).
- b. Describir las medidas aplicadas y cómo fortalecen la seguridad de su código.

El documento debe ser claro, con redacción propia y evitar copiar/pegar código en el texto o uso de alguna IA (Se revisará que no realicen uso mismo de las herramientas).